

[Chap 12] 인공신경망 구현과 KERAS 패키지



Keras 패키지의 설치

- Keras
 - 인공신경망과 딥러닝을 구현하기 위한 패키지
 - 원래는 구글의 별도 프로젝트였으나 지금은 텐서플로우와 함께 배포
- Keras의 설치
 - pip install keras (Mac: pip3 install keras)



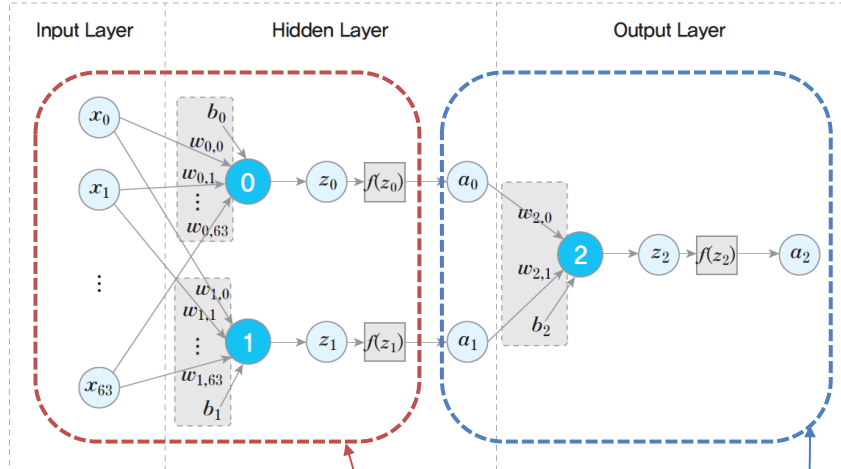
```
명령 프롬프트
Microsoft Windows [Version 10.0.19044.1826]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>pip install keras_
```

[그림 12-1] pip를 이용한 Keras 설치

손글씨 홀수/짝수 구분 인공지능망 구현

- 인공지능망 구조 (11장과 동일)



[그림 12-2] Keras로 구현하려는 인공지능망 구조

- 위의 신경망을 Keras로 구현

```
1 model = keras.Sequential([
2     keras.layers.Dense(2, activation='sigmoid', input_shape=(64,)),
3     keras.layers.Dense(1, activation='sigmoid')
4 ])
```

3

손글씨 홀수/짝수 구분 인공지능망 (계속)

- Keras 모델링 설명

```
1 model = keras.Sequential([
2     keras.layers.Dense(2, activation='sigmoid', input_shape=(64,)),
3     keras.layers.Dense(1, activation='sigmoid')
4 ])
```

- `keras.Sequential()` 내에 신경망의 각 계층을 순차적으로 리스트로 기입
 - 여러 줄로 되어 있지만, `keras.Sequential()` 안에 리스트가 하나 들어있고, 이 리스트에 두 개의 값이 있는 형태임
- (2행) 은닉층 모델링
 - 은닉층에 2개의 뉴런이 있음을 표시
 - 입력층은 `input_shape` 옵션으로 처리
 - 입력층에 64개의 값이 있음을 설정
 - `Dense`는 이전 계층과 각 노드간이 서로 모두 연결되도록 함
 - 활성화함수는 `sigmoid` 함수를 사용하도록 설정
- (3행) 출력층 모델링
 - 뉴런 1개로 구성
 - 활성화함수는 `sigmoid`
 - 앞 계층과 모두 연결

4

손글씨 홀수/짝수 구분 인공지능망 (계속)

- 손실함수와 최적화엔진 설정

```
model.compile(optimizer='SGD', loss='binary_crossentropy')
```

- model.compile() 내에 손실함수 및 최적화엔진 설정
- loss 옵션: 손실함수 설정
 - 손실함수를 엔트로피 함수인 binary_crossentropy로 설정
 - 손실함수의 종류와 사용기준은 뒤에서 설명
- optimizer 옵션: 최적화 엔진 설정
 - 최적화 엔진은 SGD(Stochastic Gradient Descent), 즉 경사하강법으로 설정

- 모델 학습

- 입력값 x_train, 출력값 y_train을 넣고 반복 에포크 횟수는 2000으로 설정

```
model.fit(x_train, y_train, epochs=2000)
```

5

손글씨 홀수/짝수 구분 인공지능망 (계속)

- 전체 코드와 실행 결과
 - 정밀도, 재현율 등의 지표가 94%에 달하고 있음
 - 앞의 장 구현 결과와 비교하여 실행 속도도 빠르고 성과도 높음
 - 최적화 알고리즘의 차이 (뒤에 설명)

```
Epoch 1/2000
45/45 [=====] - 0s 488us/step - loss: 0.6999
Epoch 2/2000
45/45 [=====] - 0s 421us/step - loss: 0.6924
Epoch 3/2000
45/45 [=====] - 0s 421us/step - loss: 0.6869
...
Epoch 2000/2000
45/45 [=====] - 0s 443us/step - loss: 0.1443
[[168 11]
 [ 10 171]]
```

	precision	recall	f1-score	support
0.0	0.94	0.94	0.94	179
1.0	0.94	0.94	0.94	181
accuracy			0.94	360
macro avg	0.94	0.94	0.94	360
weighted avg	0.94	0.94	0.94	360

예제 ex12-1.py

```
1 import numpy as np
2 from tensorflow import keras
3 from sklearn import datasets
4
5 dataset = datasets.load_digits()
6 x_data = (dataset.data/15.0).astype(np.float32)
7 y_data = (dataset.target % 2).astype(np.float32)
8
9 from sklearn.model_selection import train_test_split
10 x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
11     test_size=0.2)
12
13 model = keras.Sequential([
14     keras.layers.Dense(2, activation='sigmoid', input_shape=(64,)),
15     keras.layers.Dense(1, activation='sigmoid')
16 ])
17 model.compile(optimizer='SGD', loss='binary_crossentropy')
18 model.fit(x_train, y_train, epochs=2000)
19
20 predictions = model.predict(x_test)
21 predLabels=[]
22 for y in predictions:
23     if y<0.5: predLabels.append(0)
24     else: predLabels.append(1)
25
26 from sklearn.metrics import confusion_matrix
27 from sklearn.metrics import classification_report
28
29 print(confusion_matrix(y_test, predLabels))
30 print(classification_report(y_test, predLabels))
```

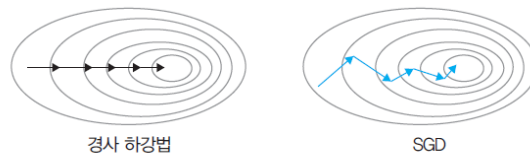
SGD (Stochastic Gradient Descent) 알고리즘

- 순수한 경사하강법 (GD: Gradient Descent)의 문제 : n 이 커지면 매우 느려짐
 - 그래디언트를 구하기 위해 n 개의 학습 데이터 전체에 대한 계산을 수행해야 함

$$\frac{\partial L}{\partial w_{31}} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) a_1(i)$$

$$\frac{\partial L}{\partial w_{11}} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) w_{31} (a_1(i) (1 - a_1(i))) x_1(i)$$

- SGD (Stochastic Gradient Descent) : 확률적 경사하강법
 - 임의의 한 건의 데이터를 샘플링하여 그래디언트를 구하고 가중치를 수정하는 것을 반복
 - 경사 방향을 정확히 계산하지는 못하지만 오래 반복하면 전체적으로 내리막 방향으로 수렴



[그림 12-3] 순수한 경사하강법(왼쪽)과 SGD(오른쪽)의 비교

- Mini Batch SGD
 - 한 건 대신, 일정 개수의 데이터를 임의로 샘플링하여 그래디언트를 계산
 - 배치(batch) : 한 번에 가중치를 계산하는데 사용하는 데이터의 양
 - Keras는 기본값으로 32로 설정

7

배치(batch)와 에포크(epoch)

- 에포크(epoch): 전체 데이터를 한 번 훑어서 지나가는 것
- 배치(batch): 한 번 그래디언트 계산에 사용하는 데이터 개수
- SGD의 배치와 에포크의 관계
 - 순수한 GD: 한 번 그래디언트 계산에 모든 데이터를 사용하므로 1 epoch에 한 번 가중치 수정 (1 epoch = 1 batch)
 - 순수한 SGD : 한 건의 데이터로 그래디언트를 계산하므로 1 epoch에 데이터 개수(n) 만큼 횟수의 가중치 수정 (1 epoch = n batch)
 - Mini batch SGD : 배치 크기(m) 만큼의 데이터로 그래디언트를 계산하므로, 1 epoch에 n/m 횟수의 가중치 수정 (1 epoch = n/m batch)

8

Keras가 지원하는 최적화 알고리즘

비선형 최적화
(nonlinear
optimization)

〈표 12-1〉 Keras의 최적화 알고리즘 종류

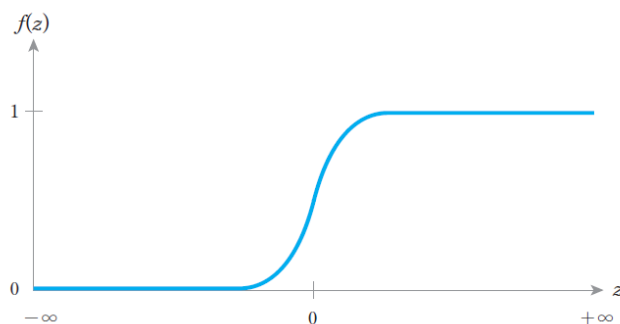
최적화 알고리즘	설명
SGD	'Stochastic Gradient Descent'. 확률적 경사하강법. 그래디언트에 학습률을 곱한 만큼 파라미터를 수정.
Adagrad	'ADAPtive GRADient algorithm'. 자주 수정된 파라미터는 학습률을 낮추고 덜 자주 수정된 파라미터에 학습률을 높이는 식으로 파라미터별로 학습률을 달리하여 수렴성을 개선. 학습률을 각 파라미터별 기존 그래디언트의 제곱합의 제곱근으로 나눔. 학습이 진행됨에 따라 학습률이 0에 가까워지는 단점이 있음
RMSprop	'Root Mean Square PROPagation'. Adagrad에서 학습률이 0에 가까워지는 되는 문제를 해결하기 위해 파라미터별 그래디언트의 제곱합 대신 지수평활법에 의한 가중이동평균의 제곱근으로 학습률을 나눔.
Adadelta	Adagrad에서 학습률이 0에 가까워지는 되는 문제를 해결하기 위해 전체 기간의 그래디언트 수정 정도 대신 최근 일정기간 동안의 그래디언트 수정 정도만 고려하도록 함
Adam	'ADAPtive Moment estimation'의 약어로서, 최근 그래디언트의 수정 정도뿐 아니라 그래디언트의 모멘텀(관성력)을 고려하여 수렴성 개선과 더불어 부분 최적에 빠지는 문제를 개선
Adamax	Adam에서 학습률 조정을 위한 계수에 L^∞ (infinity norm), 즉 벡터의 최대값을 사용하도록 변형. 임베딩(embedding) 등 일부 응용에서 Adam보다 우수한 성능을 보임.
Nadam	Adam에 NAG(Nesterov Accelerated Gradient)를 적용한 방법. 여기서 NAG는 모멘텀을 반영할 때 파라미터의 새로운 값 지점의 그래디언트를 함께 고려하여 수렴 안정성을 향상시키는 기법임.

9

활성화 함수의 종류

- 시그모이드 함수 (activation='sigmoid')
 - $(-\infty, \infty) \rightarrow (0, 1)$
 - 이진 분류(binary classification)에서 사용

$$f(z) = \frac{1}{1 + e^{-z}}$$

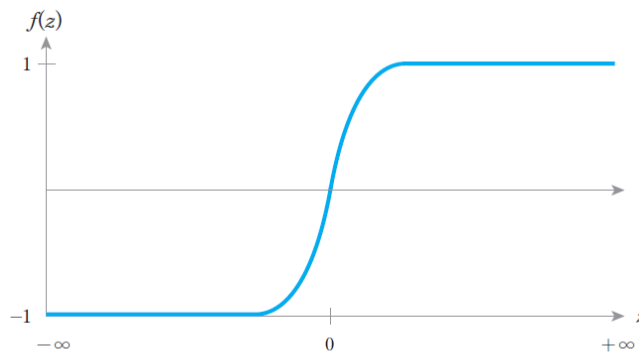


[그림 12-4] 시그모이드 활성화 함수

활성화 함수의 종류 (계속)

- 하이퍼볼릭 탄젠트(hyperbolic tangent) 함수 (activation='tanh')
 - $(-\infty, \infty) \rightarrow (-1, 1)$
 - 이진 분류(binary classification)에서 사용
 - 시그모이드와 용도가 비슷하지만, 실제 학습시에 tanh가 조금 더 잘 수렴

$$f(z) = \tanh(z) = \frac{1 - e^{-2z}}{1 + e^{-2z}}$$



[그림 12-5] 하이퍼볼릭 탄젠트 활성화 함수

11

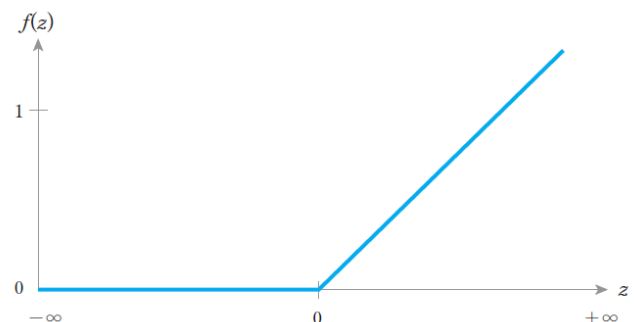
활성화 함수의 종류 (계속)

- 그래디언트 소실 문제(gradient vanishing problem)
 - z가 작은 값이나 큰 값이 되면 급격히 기울기가 낮아지면서 기울기가 거의 0이 되어 학습이 잘 안되는 현상
 - 시그모이드 함수와 하이퍼볼릭 탄젠트 함수의 공통적 문제

- ReLU (Rectified Linear Unit) 함수 (activation='relu')

- 그래디언트 소실 문제 해결
- z가 양수이면 그대로 보내고 음수는 0으로 변환
 - $(-\infty, \infty) \rightarrow (0, \infty)$
- 장점
 - 양수 구간에서 그래디언트를 보존하여 그래디언트 소실 문제 해결
 - 선형함수 형태여서 계산 속도가 빠름
- 딥러닝의 은닉층은 거의 relu를 사용

$$f(z) = \max(0, z)$$



[그림 12-6] ReLU 활성화 함수

12

활성화 함수의 종류 (계속)

- Softmax 함수
 - 다중 클래스 분류(multi-class classification)에 사용
 - 출력층에 여러 개의 뉴런이 있음
 - 각각의 뉴런의 가중합을 z_1, z_2, z_3 라 할 때,

$$f(z_1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$f(z_2) = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$f(z_3) = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

13

Softmax 함수: 강아지, 고양이, 곰돌이 판별 문제

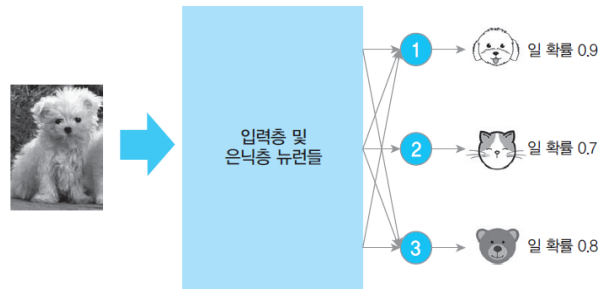
- 1번 사진에서 $z_1 = 2.2, z_2 = 0.85, z_3 = 1.39$ 라면, 시그모이드 함수 적용시

$$f(z_1) = \frac{1}{1+e^{-2.2}}=0.9, \quad f(z_2) = \frac{1}{1+e^{-0.85}}=0.7, \quad f(z_3) = \frac{1}{1+e^{-1.39}}=0.8$$

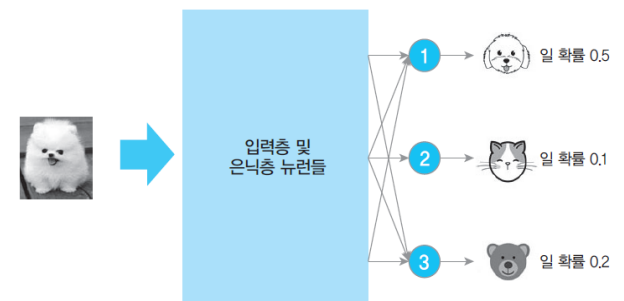
- 2번 사진에서 $z_1 = 0, z_2 = -2.19, z_3 = -1.372$ 라면, 시그모이드 함수 적용시

$$f(z_1) = \frac{1}{1+e^0}=0.5, \quad f(z_2) = \frac{1}{1+e^{2.19}}=0.1, \quad f(z_3) = \frac{1}{1+e^{1.37}}=0.2$$

- 시그모이드 단순 적용의 문제점
 - 1번 사진에서는 강아지와 곰돌이를 혼동하고 있으나, 2번에서는 강아지일 확률이 다른 것보다 훨씬 높음
 - 그러나, 확률값 자체는 1번의 강아지 확률이 높음
 - 단순히 각 뉴런의 출력값만으로 도출된 판단의 확신 정도를 산정하는 것은 문제가 있고, 해당 판단의 확률이 다른 판단의 확률과 얼마나 차이가 있는지를 반영하는 확률 계산이 필요



[그림 12-7] 강아지 사진 판별 예1



[그림 12-8] 강아지 사진 판별 예 2

Softmax 함수(계속)

- 1번 사진과 2번 사진의 softmax 함수의 결과값 비교

$$\begin{aligned}f(z_1) &= \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^{2.2}}{e^{2.2} + e^{0.85} + e^{1.39}} = 0.59 & f(z_1) &= \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^0}{e^0 + e^{-2.19} + e^{-1.37}} = 0.73 \\f(z_2) &= \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^{0.85}}{e^{2.2} + e^{0.85} + e^{1.39}} = 0.15 & f(z_2) &= \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^{-2.19}}{e^0 + e^{-2.19} + e^{-1.37}} = 0.08 \\f(z_3) &= \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^{1.39}}{e^{2.2} + e^{0.85} + e^{1.39}} = 0.26 & f(z_3) &= \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^{-1.37}}{e^0 + e^{-2.19} + e^{-1.37}} = 0.19\end{aligned}$$

- 두 번째 경우에서 강아지 확률이 더 높게 나타남을 확인

- Softmax 함수는 여러 개의 뉴런 출력값을 종합해서 상대적 확신 차이를 반영
 - 전체 확률의 합이 1이 되게 하면서 상대적인 확률의 차이를 반영한 확신의 정도를 계산해주는 장점
 - 출력층에 두 개 이상 뉴런이 있는 다중 분류 문제의 활성화 함수로 일반적 활용

15

손실 함수(loss function)

- mean_square_error: 평균제곱오차

$$Loss = \frac{1}{n} \sum_{i=1}^n (y_i - a_i)^2$$

- binary_crossentropy: 이진 크로스 엔트로피
 - 이진 분류에서 사용

$$Loss = -\frac{1}{n} \sum_{i=1}^n \{y_i \log(a_i) + (1 - y_i) \log(1 - a_i)\}$$

- categorical_crossentropy: 범주형 크로스 엔트로피
 - 다중 분류(분류 대상 클래스가 3개 이상)에서 사용

$$Loss = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c y_{ij} \log(a_{ij})$$

- 예) 3 클래스 분류. 데이터 2건에 대해 다음을 가정
 - 정답값: (1,0,0), (1,0,0)
 - 예측값: (0.59, 0.15, 0.26), (0.73, 0.08, 0.19)
 - 범주형 크로스 엔트로피는 다음과 같이 계산됨

$$\begin{aligned}Loss &= -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c y_{ij} \log(a_{ij}) = -\frac{1}{2} \sum_{i=1}^2 \sum_{j=1}^3 y_{ij} \log(a_{ij}) \\&= -\frac{1}{2} [\{1 \cdot \log(0.59) + 0 \cdot \log(0.15) + 0 \cdot \log(0.26)\} \\&\quad + \{1 \cdot \log(0.73) + 0 \cdot \log(0.08) + 0 \cdot \log(0.19)\}] \\&= 0.4212\end{aligned}$$

16

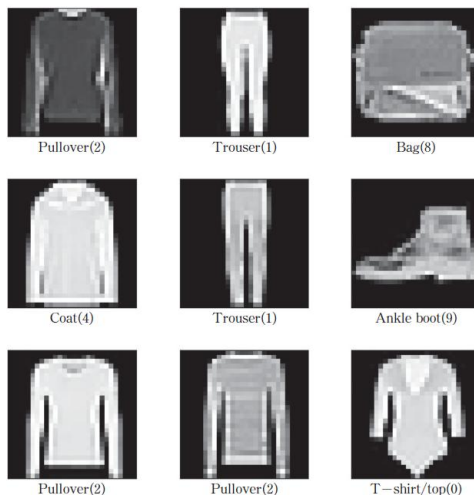
손실 함수 (계속)

- `sparse_categorical_crossentropy` : 희소 범주형 크로스 엔트로피
 - `categorical_crossentropy`와 계산식은 동일하나, 정답값 데이터 형태의 표현 방식에 따라 구분하여 사용
 - 정답값이 (1, 0, 0), (0, 1, 0), (0, 0, 1)과 같이 각 정답값이 클래스 개수만큼의 요소를 가지는 벡터로 표현되면 `categorical_crossentropy`를 사용
 - 정답값 표현에 메모리 사용량이 큰 문제
 - 정답값이 0, 1, 2처럼 정수 숫자로 표현되면 `sparse_categorical_crossentropy` 사용
 - 메모리 절약 가능

17

이미지 인식 예제: Fashion MNIST

- 각 데이터포인트는 흑백으로 된 의류 사진과, 이 의류가 10가지의 구분 중에 어떤 것인지를 나타내는 숫자로 구성
 - 각각의 이미지는 28×28 픽셀, 각 픽셀은 0에서 255 사이의 밝기 값
- 60,000개의 학습용 데이터와 10,000개의 테스트 데이터



[그림 12-9] Fashion MNIST의 이미지 샘플

<표 12-2> Fashion MNIST 데이터셋의 정답값 의미

정답 라벨	의미
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

데이터 살펴보기

- Fashion MNIST 데이터를 읽어 데이터 형태와 첫번째 이미지 출력

예제 ex12-2.py

```
1 from tensorflow import keras
2 fashion_mnist = keras.datasets.fashion_mnist
3 (train_images, train_labels), (test_images, test_labels) = \
4     fashion_mnist.load_data()
5 print(train_images.shape)
6 print(train_labels.shape)
7 print(test_images.shape)
8 print(test_labels.shape)
9
10 print('Train label 0 = ', train_labels[0])
11 print(train_images[0])
12
13 import matplotlib.pyplot as plt
14 plt.imshow(train_images[0])
15 plt.colorbar()
16 plt.show()
```

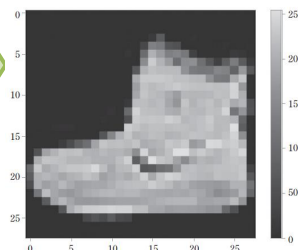
(2~4행) 데이터 읽어오기

(5~8행) 학습 및 테스트
데이터 형태 출력

(10~11행) 0번 데이터 값 출력

(13~16행) 0번 데이터
이미지 출력

- (60000, 28, 28)
- (60000,)
- ▶ (10000, 28, 28)
- (10000,)

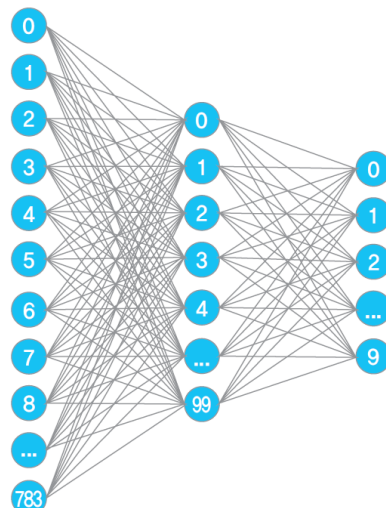
[illegible]

[그림 12-10] 학습용 데이터 0번의 이미지

이미지 인식 인공지능경망 구현

- 신경망 구조 설계
 - 입력층: 가장 단순하게 784(= 28×28)개 픽셀 값
 - 입력값을 255로 나누어 0과 1 사이 값으로 변환
 - 은닉층: 계층 수와 각 계층의 뉴런 개수를 정하는 것이 관건이나, 여기서는 단일 계층으로 뉴런 갯수 100개로 설정
 - 출력층: 구분하려는 클래스의 수만큼의 10개의 뉴런 필요
 - 각각의 뉴런은 각각에 대응되는 라벨에 대해, 입력된 이미지가 해당 라벨인지 아닌지를 판별

```
1 train_images = train_images/255.0
2 test_images = test_images/255.0
```



```
1 model = keras.Sequential([
2     keras.layers.Flatten(input_shape=(28, 28)),
3     keras.layers.Dense(100, activation='relu'),
4     keras.layers.Dense(10, activation='softmax')
5 ])
```

[그림 12-11] Keras로 구현하려는 인공신경망 구조

이미지 인식 인공지능경망 구현 (계속)

- 최적화 알고리즘과 손실함수 설정

```
model.compile(optimizer = 'adam', loss = 'sparse_categorical_crossentropy')
```

- 학습용 이미지와 정답을 주고 학습 진행

```
model.fit(train_images, train_labels, epochs=1)
```

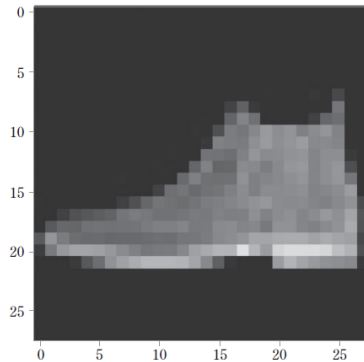
- 테스트 데이터를 적용하여 예측

```
predictions = model.predict(test_images)
```

- 첫 번째 테스트 이미지와 예측값 출력

```
1 import matplotlib.pyplot as plt
2 print('Test label 0 =', test_labels[0])
3 print('Prediction 0 =', predictions[0])
4 plt.imshow(test_images[0])
5 plt.show()
```

```
Test label 0 = 9
Prediction 0 = [2.6515195e-06 1.5080832e-07 3.8006117e-06 1.7151444e-06 2.2985461e-05
3.2011930e-02 4.0852105e-06 3.1448203e-01 3.7465435e-03 6.4972407e-01]
```



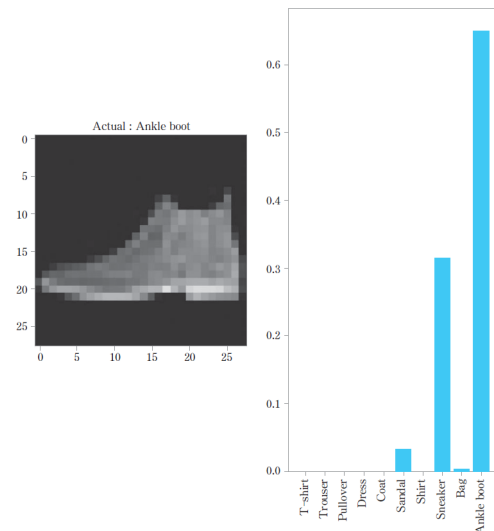
[그림 12-12] 테스트 이미지 0번

21

이미지 인식 인공지능경망 구현 (계속)

- 각 뉴런 예측값 막대그래프로 출력

```
1 import matplotlib.pyplot as plt
2 names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
3         'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
4 for i in range(30):
5     print('Actual : ', test_labels[i], names[test_labels[i]])
6     plt.subplot(1,2,1)
7     plt.imshow(test_images[i])
8     plt.title('Actual : '+names[test_labels[i]])
9     plt.subplot(1,2,2)
10    plt.bar(names, predictions[i])
11    plt.xticks(rotation='vertical')
12    plt.show()
```



[그림 12-13] 테스트 이미지 0번에 대한 예측

22

성능 평가

- 현재 predictions 변수에는 각 데이터별로 10개씩의 확률값이 들어가 있음
 - 가장 확률이 높게 나타난 번호를 예측값으로 사용하기 위해 predictions의 각 벡터에 대해서 np.argmax를 이용해 가장 높은 값이 나타난 인덱스를 찾아서 이 값을 예측값으로 저장

```
1 import numpy as np
2 predicted_labels = []
3 for p in predictions:
4     predicted_labels.append(np.argmax(p))
```

- 예측 성능지표 출력

```
1 from sklearn.metrics import classification_report
2 print(classification_report(test_labels, predicted_labels))
```

	precision	recall	f1-score	support
0	0.88	0.78	0.82	1000
1	0.99	0.96	0.98	1000
2	0.73	0.81	0.77	1000
3	0.83	0.93	0.88	1000
4	0.87	0.63	0.73	1000
5	0.98	0.96	0.97	1000
6	0.63	0.76	0.69	1000
7	0.93	0.97	0.95	1000
8	0.98	0.96	0.97	1000
9	0.96	0.95	0.96	1000
accuracy			0.87	10000
macro avg	0.88	0.87	0.87	10000
weighted avg	0.88	0.87	0.87	10000